

```

void CreateTreeNS(int n)
{
    int i, temp;
    for (i = 1; i <= n; i++)
    {
        // temp = 1 + rand() % 99; // random difference number
        // 1..99
        temp = i;
        data[i] = temp;
    }
}

```

ใช้สำหรับสร้างข้อมูลต้นไม้ (Tree) ในรูปแบบ Node Sequence Method โดยเก็บข้อมูลลงใน Array data[]

```

void ShowArray()
{
    int i = 1;
    while (data[i] != 0)
    {
        printf("[%i]%d \t", i, data[i]);
        i++;
        if((i - 1) % 3 == 0) { // Show every 3 array[] then new
            line
            printf("\n");
        }
    }
    printf("\n===== \n");
}

```

ใช้แสดงข้อมูลภายใน Array data[]

```

void ShowTree()
{
    int j, level, start, ends;
    j = 1;
    level = 1; // Start at Level 1
    printf("\n");
    while (data[j] != 0)
    {

```

```

        start = pow(2, level) / 2; // Calculate START Node of
this level
        ends = pow(2, level) - 1; // Calculate END Node of this
level
        for (j = start; j <= ends; j++)
        {
            if (data[j] != 0)
            {
                switch (level)
                {
                    case 1:
                        printf("%40d", data[j]);
                        break;
                    case 2:
                        if (j == 2)
                            printf("%20d", data[j]);
                        else
                            printf("%40d", data[j]);
                        break;
                    case 3:
                        if (j == 4)
                            printf("%10d", data[j]);
                        else
                            printf("%20d", data[j]);
                        break;
                    case 4:
                        if (j == 8)
                            printf("%5d", data[j]);
                        else
                            printf("%10d", data[j]);
                        break;
                    case 5:
                        if (j == 16)
                            printf("%d", data[j]);
                        else
                            printf("%5d", data[j]);
                        break;
                }
            }
        }
        printf("\n\n"); // Line feed
        level++;

```

```

    }
}

```

ใช้แสดงโครงสร้างต้นไม้ในลักษณะ Tree Diagram

```

void PreOrder(int i)
{
    int info, lson, rson;
    info = data[i];    // Root info
    if (info != 0) // if INFO NOT 0
    {
        printf(" %d", data[i]); // Display INFO
        lson = 2 * i;           // Calculate LSON
        rson = 2 * i + 1;       // Calculate RSON
        PreOrder(lson);         // Call left Son by PreOrder
        PreOrder(rson);         // Call Right Son by PreOrder
    }
}

```

ใช้ Traversal แบบ PreOrder

```

void InOrder(int i)
{
    int info, lson, rson;
    info = data[i];    // Root info
    if (info != 0) // if INFO NOT 0
    {
        lson = 2 * i;           // Calculate LSON
        rson = 2 * i + 1;       // Calculate RSON
        InOrder(lson);         // Call left Son by InOrder
        printf(" %d", data[i]); // Display INFO
        InOrder(rson);         // Call Right Son by InOrder
    }
}

```

ใช้ Traversal แบบ InOrder

```

void PostOrder(int i)
{
    int info, lson, rson;
    info = data[i];    // Root info
    if (info != 0) // if INFO NOT 0
    {
        lson = 2 * i;           // Calculate LSON
        rson = 2 * i + 1;       // Calculate RSON
        PostOrder(lson);        // Call left Son by PostOrder
        PostOrder(rson);        // Call Right Son by PostOrder
        printf(" %d", data[i]); // Display INFO
    }
}

```

🔗 Traversal แบบ PostOrder

```

int main()
{
    N = 31;
    CreateTreeNS(N); // Create N Node
    while (ch != 'E')

    {
        printf("\nTREE (NODE SEQUENCE)\n");
        printf("=====\n");
        ShowArray();
        ShowTree();
        printf("\nMENU => P:PreOrder I:InOrder O:PostOrder
E:Exit");
        printf("\n-----
\n");
        //ch = getch();
        scanf(" %c", &ch); // GCC/MinGW not working for conio.h
        //getch()
        switch (ch)
        {
            case 'P':
                ShowTree();
                printf("PRE ORDER TRAVERSAL : ");
                PreOrder(1);

```

```

        printf("\n"); // Line feed
        break;
    case 'I':
        ShowTree();
        printf("IN ORDER TRAVERSAL : ");
        InOrder(1);
        printf("\n"); // Line feed
        break;
    case 'O':
        ShowTree();
        printf("POST ORDER TRAVERSAL : ");
        PostOrder(1);
        printf("\n"); // Line feed
        break;
    } // End Switch...case
} // End While
return (0);
} // End MAIN

```

เป็นจุดเริ่มต้นของโปรแกรม และความคุมการทำงานทั้งหมด

และเรียงลำดับการทำงานตามแต่ละฟังก์ชันดังกล่าว

ขั้นตอนที่ 1 สร้างต้นไม้ สร้าง Complete Binary Tree จำนวน 31 Node

ขั้นตอนที่ 2 แสดงผลต้นไม้ แสดงข้อมูลใน Array และแสดงรูปแบบ Tree

ขั้นตอนที่ 3 รับคำสั่งจากผู้ใช้ รับตัวอักษรจากคีย์บอร์ด

ขั้นตอนที่ 4 เลือกการ Traversal เริ่ม Traversal จาก Root

ขั้นตอนที่ 5 จบโปรแกรม เมื่อผู้ใช้ป้อน E จะเป็นเท็จและออกจากลูป

```

Node *Allocate()
{ // Allocate 1 node from storage pool and return
    struct Node *temp;
    temp = (Node *)malloc(sizeof(Node)); // Allocate node by
size declare
    return (temp);
}

```

ใช้สำหรับจองหน่วยความจำ (Memory Allocation) ให้กับ Node ใหม่ในต้นไม้ ทำหน้าที่สร้าง Node ใหม่และคืน Address ของ Node ให้โปรแกรมนำไปใช้งานต่อ

```

void CreateTreeNP(int n)
{

```

```

int i, temp, Father;

struct Node *p, *FatherPT;
T = NULL; // Set start of T Pointer
for (i = 1; i <= n; i++)
{
    p = Allocate(); // Allocate NODE p
    // temp = 1 + rand() % 99; // random difference number
    1..99
    temp = i;
    info[i] = temp; // Keep data for Check
    Correcting of Sequence
    address[i] = p; // Keep Address of Node
    Sequence
    p->info = temp; // Keep data to INFO
    p->lson = NULL; // Set default LSON=NULL
    p->rson = NULL; // Set default RSON=NULL
    if (T == NULL)
    {
        // Check for T=NULL?
        T = p; // Set T point to first node for begining
    of Treer
    }
    else
    {
        Father = (i / 2); // Calculate FATHER
        FatherPT = address[Father]; // Get pointer of
        Father Node
        if (FatherPT->lson == NULL) {
            FatherPT->lson = p; // Link LSON to new node
        }
        else {
            FatherPT->rson = p; // link RSON to new node
        }
    }
}
}

```

ใช้สร้าง Binary Tree ด้วยวิธี Node Pointer Method

```

void ShowTree()
{
    int j, level, start, ends;

```

```

j = 1;
level = 1; // Start at Level 1
printf("\n");
while (info[j] != 0)
{
    start = pow(2, level) / 2; // Calculate START Node of
this level
    ends = pow(2, level) - 1; // Calculate END Node of
this level
    for (j = start; j <= ends; j++)
        if (info[j] != 0)
        {
            switch (level)
            {
                case 1:
                    printf("%40d", info[j]);
                    break;
                case 2:
                    if (j == 2)
                        printf("%20d", info[j]);
                    else
                        printf("%40d", info[j]);
                    break;
                case 3:
                    if (j == 4)
                        printf("%10d", info[j]);
                    else
                        printf("%20d", info[j]);

                    break;
                case 4:
                    if (j == 8)
                        printf("%5d", info[j]);
                    else
                        printf("%10d", info[j]);
                    break;
                case 5:
                    if (j == 16)
                        printf("%d", info[j]);
                    else
                        printf("%5d", info[j]);
                    break;
            }
        }
}

```

```

    }
    }
    printf("\n"); // Line feed
    level++;
}
}

```

ใช้แสดงโครงสร้างต้นไม้ในรูปแบบ Tree Diagram

```

void PreOrder(struct Node *i)
{
    if (i != NULL)
    {
        // if i NOT NULL
        printf(" %d", i->info); // Display INFO
        PreOrder(i->lson);      // Call left Son by PreOrder
        PreOrder(i->rson);      // Call Right Son by PreOrder
    }
}

```

ใช้ Traversal แบบ PreOrder

```

void InOrder(struct Node *i)
{
    if (i != NULL)
    {
        // if i NOT NULL
        InOrder(i->lson);      // Call left Son by InOrder
        printf(" %d", i->info); // Display INFO
        InOrder(i->rson);      // Call Right Son by InOrder
    }
}

```

ใช้ Traversal แบบ InOrder


```

void PostOrder(struct Node *i)
{
    if (i != NULL)
    {
        // if i NOT NULL
        PostOrder(i->lson); // Call left Son by PostOrder
        PostOrder(i->rson); // Call Right Son by PostOrder
        printf(" %d", i->info); // Display INFO
    }
}

```

🔗 Traversal แบบ PostOrder

```

int main()
{
    N = 31;
    CreateTreeNP(N);
    while (ch != 'E') // Loop until 'E' Pressed
    {
        printf("\nPROGRAM TREE(Node Pointer) \n");
        printf("===== \n");
        printf("N : %d\n", N);
        printf("Sequence of data : \n");
        for (i = 1; i <= N; i++) { // Show Data Sequence
            printf("%d \t", info[i]);
            if(i % 5 == 0) { // Show every 5 array[] then new
line
                printf("\n");
            }
        }
        ShowTree(); // Show tree structure
        printf("\nMENU => P:PreOrder I:InOrder O:PostOrder
E:Exit");
        printf("\n-----
---\n");
        //ch = getch();
        scanf(" %c", &ch); // GCC/MinGW not working for
conio.h -> getch()
        switch (ch)
        {

```

```

        case 'P':
            printf("PRE ORDER TRAVERSAL : ");
            PreOrder(T);
            printf("\n-----
-----\n");
            break;
        case 'I':
            printf("IN ORDER TRAVERSAL : ");
            InOrder(T);
            printf("\n-----
-----\n");
            break;
        case 'O':
            printf("POST ORDER TRAVERSAL : ");
            PostOrder(T);
            printf("\n-----
-----\n");
            break;
    } // End Switch...case
} // End While
return (0);
} // End MAIN

```

เป็นจุดเริ่มต้นและควบคุมการทำงานทั้งหมดของโปรแกรม

ขั้นตอนที่ 1 สร้างต้นไม้ สร้างต้นไม้จำนวน 31 Node

ขั้นตอนที่ 2 แสดงข้อมูลลำดับ Node แสดงข้อมูลที่เก็บอยู่ในต้นไม้

ขั้นตอนที่ 3 แสดงโครงสร้างต้นไม้ แสดง Tree Diagram

ขั้นตอนที่ 4 รับคำสั่งจากผู้ใช้ คำสั่งที่รองรับ P,I,O และ E

ขั้นตอนที่ 5 เลือกการทำงานตามเมนู PreOrder(T); , InOrder(T); , PostOrder(T);

ขั้นตอนที่ 6 จบโปรแกรม เมื่อผู้ใช้ป้อน E จะสิ้นสุดการทำงาน